

FUNDAMENTOS DE ARQUITECTURA LLM · SESIÓN 2

Tipos de modelos y escenarios de aplicación

Modelos propietarios vs. open-source · API gestionada vs. self-hosted · Casos de uso empresariales

Qué vamos a cubrir hoy

01

Modelos propietarios vs. open-source

Familias destacadas y criterios de elección

02

API gestionada vs. self-hosted

Qué implica cada modalidad de consumo

03

Casos de uso empresariales

Dónde encaja cada modalidad, con el caso TechCorp

04

Actividad práctica

Comparar un mismo prompt en ambas modalidades, en vivo

TEMA 1

Modelos propietarios vs. open-source

Qué distingue a cada tipo, familias destacadas y cuándo elegir cada uno

¿Qué distingue a cada tipo?

	PROPIETARIOS	OPEN-SOURCE
Acceso a los pesos	No — solo vía API	Sí — se descargan y ejecutan
Dónde corre	Infraestructura del proveedor	Donde tú decidas
Modelo de costo	Pago por token consumido	Sin licencia; costo = tu infraestructura
Personalización	Limitada (fine-tuning gestionado)	Total (fine-tuning, cuantización, poda)
Control de datos	Depende de la política del proveedor	Total, si tú decides el despliegue

Familias destacadas (2025–2026)

PROPIETARIOS

- › OpenAI — familia GPT (GPT-4o, GPT-4o-mini, o-series)
- › Anthropic — familia Claude (Opus, Sonnet, Haiku)
- › Google — familia Gemini (Pro, Flash)
- › Acceso vía API directa o hyperscaler (Azure, Bedrock, Vertex AI)

OPEN-SOURCE / OPEN-WEIGHT

- › Meta — familia Llama (incluye llama3.2:3b, usado en esta clase)
- › Mistral AI — Mistral / Mixtral (densos y mezcla de expertos)
- › Alibaba — familia Qwen, fuerte desempeño multilingüe
- › DeepSeek — orientados a razonamiento y código

Nota de clase: las familias cambian rápido — lo importante es saber qué criterios aplicar, no memorizar el ranking del momento.

Criterios para elegir entre ambos

1

Sensibilidad de los datos

¿Pueden salir del perímetro de la empresa?

2

Volumen de uso esperado

A muy alto volumen, el costo por token pesa más

3

Personalización requerida

Fine-tuning profundo solo es viable con acceso a los pesos

4

Capacidad interna del equipo

Operar self-hosted en producción requiere MLOps/Infra

5

Velocidad de llegada al mercado

API gestionada permite prototipar en días

6

Complejidad de la tarea

Tareas acotadas rinden bien con modelos pequeños open-source

TEMA 2

Modalidades de consumo: API gestionada vs. self-hosted

Qué implica operar cada modalidad, en costo, control y responsabilidad

Dos formas de consumir un LLM

SH

Self-hosted

- › El modelo corre en tu propia infraestructura
- › Sin costo por token — el costo es hardware, energía y operación
- › Control total: los datos nunca salen de tu entorno
- › Tú eres responsable de disponibilidad y escalamiento
- › Ejemplo en clase: Ollama + llama3.2:3b, local

AG

API gestionada

- › No administras infraestructura ni el modelo
- › Pagas por token consumido (entrada + salida)
- › El proveedor resuelve disponibilidad y actualizaciones
- › Tus datos viajan a la infraestructura del proveedor
- › Ejemplo en clase: OpenAI / Azure OpenAI, gpt-4o-mini

Lo que vamos a correr hoy



El mismo prompt se envía por el mismo endpoint. El backend decide, según el "provider" recibido, si lo resuelve localmente (Ollama) o llamando a una API gestionada — así comparamos ambas modalidades con una sola pieza de código.

TEMA 3

Casos de uso empresariales comunes

Dónde encaja cada modalidad — con el caso TechCorp Latinoamérica

Seis escenarios, dos modalidades

Asistente de soporte interno de TI	API gestionada
Análisis de contratos y datos de proveedores	Self-hosted
Chatbot de atención al cliente (sitio público)	API gestionada
Resumen de reportes de mantenimiento de planta	Self-hosted
Borradores de comunicados corporativos	API gestionada
Clasificación de tickets de mesa de ayuda	Self-hosted

Manos a la obra

1

Levanta Ollama y el backend FastAPI siguiendo el README del repositorio

2

Abre frontend/index.html y ejecuta el mismo prompt contra ambos proveedores

3

Completa la tabla comparativa (latencia, costo, calidad) en docs/03_casos_uso_empresariales.md

4

Elige un escenario de TechCorp y justifica qué modalidad usarías, y por qué

RECAPITULACIÓN

Lo esencial de hoy

- › Propietario vs. open-source: la diferencia clave es el acceso a los pesos del modelo
- › Self-hosted vs. gestionada: control y costo de infraestructura vs. pago por token y cero operación
- › La modalidad correcta depende del caso: sensibilidad de datos, volumen, capacidad interna
- › Próxima sesión: profundizamos en los componentes internos de la arquitectura LLM